Plateforme de marché financier simulé

1 Contexte et périmètre

Une salle de marché pédagogique suit vingt actions. Elle veut observer des échanges, comprendre les prix et repérer des mouvements inhabituels. Le projet construit une plateforme de données. Il ne cherche pas à prédire les cours ni à proposer des décisions d'investissement.

Les symboles sont ceux d'entreprises réelles. Les prix, les quantités et les transactions sont entièrement simulés. Ce choix conserve le générateur fourni et répond à la demande de travail. Le sujet initial parle d'actions fictives. Cette adaptation est donc explicitée dans le dépôt et dans l'application.

La plateforme comprend un chemin continu et un chemin planifié. Le chemin continu transporte les échanges et calcule des chandeliers. Le chemin planifié construit les indicateurs analytiques et vérifie leur qualité. Les deux chemins partagent les mêmes définitions métier et le même fuseau UTC.

Les exigences obligatoires FIN-01 à FIN-08 constituent la priorité. Elles couvrent le débit, Kafka, Spark, Iceberg, Gold, les contrôles, Grafana et les alertes Streamlit. La latence de bout en bout et ClickHouse sont des options. Elles ne sont pas présentées comme réalisées.

Le matériel cible dispose de seize gigaoctets de mémoire. Docker Desktop exécute des conteneurs Linux. Les images préparent leurs dépendances pendant la construction. Les volumes conservent les journaux, les tables et les checkpoints. Le démarrage doit rester possible avec deux commandes depuis le dossier du projet.

Les auteurs doivent renseigner les membres du groupe dans le README. Aucun nom ni résultat de soutenance n'a été inventé.

2 Examen initial et architecture

Le dépôt initial contient un générateur Python, un producteur Kafka et un consommateur Bronze. Le stockage utilise une API S3 et conserve du JSONL et du Parquet. Le topic trades possède six partitions. Le symbole constitue la clé des messages. Ces éléments sont réutilisés.

L'exemple Spark initial calcule des chandeliers mais ne déploie pas de catalogue ni de sorties persistantes. PostgreSQL, dbt, DuckDB, Soda, Dagster et les interfaces sont absents. Le test du choc utilise encore FIN01 alors que le générateur emploie AAPL. Le README ne correspond plus aux symboles du code. Ces écarts sont corrigés.

Le chemin continu devient : générateur, Kafka, Spark, PostgreSQL et interfaces. En parallèle, le consommateur Bronze archive les mêmes messages. Spark lit directement Kafka et n'attend jamais la fin d'un lot Bronze.

Le chemin analytique devient : Iceberg Silver, export cohérent, DuckDB, modèles dbt, contrôles Soda et publication Gold. Dagster planifie ce chemin. Les services continus ne dépendent pas de ses horaires.

Hive Metastore référence les tables Iceberg. PostgreSQL héberge une base distincte pour ses métadonnées. MinIO conserve les fichiers de données et de métadonnées Iceberg. Le rôle de chaque outil reste séparé et justifié.

Le schéma Mermaid du README montre les deux chemins et les trois couches. Le dictionnaire documente le grain de chaque table. Le tableau de couverture relie chaque exigence au code et à sa vérification. Il distingue une fonctionnalité écrite d'un résultat effectivement exécuté.

3 Génération et transport Kafka

Le générateur parcourt les vingt symboles de façon cyclique. Chaque cycle garantit leur présence. Les prix suivent une petite marche aléatoire multiplicative. Une graine fixe stabilise les tirages. Chaque transaction reçoit un identifiant UUID distinct et une heure UTC avec microsecondes.

Le champ quantity désigne le nombre d'actions échangées. Le champ volume est réservé aux agrégats de chandeliers. Les anciens messages utilisent volume pour une transaction ; les lecteurs gardent cet alias pour préserver la compatibilité avec les données déjà stockées.

La cible est de 220 transactions par seconde au total. Elle laisse une marge par rapport au minimum de 200. L'horloge monotone pilote les envois. Les statistiques utilisent les callbacks de livraison Kafka. Un simple appel produce ne constitue pas une preuve de livraison.

Le producteur active l'idempotence et demande tous les acquittements disponibles. Le symbole est la clé de partitionnement. À nombre de partitions constant, les événements d'une action arrivent dans une même partition. Kafka conserve cet ordre local. Le calcul OHLC utilise toutefois l'heure métier et un départage par identifiant.

La configuration possède un seul broker. Le facteur de réplication vaut un. Elle répond à une maquette sur ordinateur personnel, sans tolérance à la perte du broker. Changer le nombre de partitions peut modifier le routage des clés. Cette opération exige donc une migration et ne doit pas être improvisée pendant la démonstration.

Le choc contrôlé augmente AAPL de huit pour cent après six minutes. Sa détection est assurée ensuite par les données observées dans PostgreSQL, indépendamment du message de log du générateur.

4 Archivage Bronze et reprise

Le consommateur Bronze utilise son propre groupe. Il accumule un lot limité par un nombre de messages ou une durée. Les limites évitent de garder indéfiniment les événements en mémoire. Un arrêt normal écrit également le dernier lot incomplet.

Le JSONL conserve les payloads bruts. Le Parquet ajoute les types et les coordonnées Kafka. Les colonnes de provenance comprennent le topic, la partition, l'offset, l'identifiant du lot et l'heure d'ingestion. Un message malformé conserve son brut et une erreur de décodage identifiable. Il ne doit pas bloquer tous les messages suivants.

Le consommateur publie le manifeste après les deux objets de données. Le manifeste indique leurs clés, leur nombre de lignes et les prochains offsets par partition. Le vérificateur l'utilise pour sélectionner un lot complet. Son utilité justifie son maintien dans le projet.

Les offsets Kafka sont validés de façon synchrone après la réussite de l'archivage. Un échec arrête le consommateur et Docker le relance. Les offsets non validés sont relus. Un crash après S3 mais avant Kafka peut donc rejouer les mêmes événements.

L'identifiant de lot dépend des coordonnées Kafka. Un lot identique retrouve les mêmes clés d'objets. Si le découpage change après reprise, les objets peuvent contenir des doublons. La garantie est at-least-once. Elle ne devient pas exactly-once par la seule présence d'un manifeste.

Le chemin batch_date reprend le plus ancien événement du lot. Il n'est pas une partition stricte de chaque ligne par date. Un lot peut traverser minuit. Les messages sans heure décodable sont classés sous invalid. La couche Silver déduplique les identifiants métier dans la borne temporelle définie.

5 Spark et temps d'événement

Spark Structured Streaming lit directement le topic trades. Le JSON est décodé avec un schéma explicite. Le prix est décimal, la quantité entière et l'heure typée. Les contrôles vérifient les champs essentiels, le symbole, la clé Kafka, la positivité et la présence d'un fuseau. Un événement trop loin dans le futur est refusé pour éviter d'avancer artificiellement le watermark.

Les rejets de contrat sont écrits dans une sortie Parquet séparée. Ils conservent le brut et les coordonnées Kafka. Les événements trop anciens abandonnés par les opérateurs temporels sont suivis par leurs métriques. Leurs messages restent auditables en Bronze.

La première requête valide et déduplique les trade_id avec un watermark de deux minutes. Elle écrit dans un Parquet intermédiaire persistant. Deux autres requêtes lisent cette zone : l'une publie les fenêtres provisoires, l'autre les fenêtres finalisées. Chaque requête possède son checkpoint. Cette séparation évite de chaîner deux opérations avec état dans une requête en mode update.

Les fenêtres durent une minute : leur début est inclus et leur fin exclue. Open et Close suivent le minimum et le maximum du couple heure et trade_id. High et Low sont les extrêmes des prix. Volume additionne les quantités. Notional additionne les produits prix et quantité.

Le watermark de deux minutes accepte des retards courts tout en bornant l'état. Il dépend des événements reçus. Il n'avance pas lorsque le flux s'arrête. Les deux étapes peuvent ajouter un délai. Une donnée admise corrige une fenêtre provisoire ; une donnée hors borne ne déclenche pas de correction historique automatique.

6 Silver Iceberg et catalogue Hive

La table lake.finance.candles contient une ligne par action et par minute. Elle conserve OHLC, volume, notional, nombre d'échanges et dernier événement. Le partitionnement Iceberg utilise le jour de minute_start. Les prix restent en décimal pour éviter une perte inutile lors des sommes pondérées.

Les chandeliers provisoires sont utiles à l'affichage. Ils sont publiés dans PostgreSQL. Silver reçoit uniquement les fenêtres finalisées après passage du watermark. La table ne reçoit donc pas chaque version provisoire en append.

Le callback final utilise un MERGE sur le symbole et la minute. Rejouer le même micro-lot remplace la même ligne au lieu d'en ajouter une autre. Cette stratégie facilite les reprises après incident et permet de comparer une clé avant et après redémarrage.

Hive Metastore conserve les définitions et pointeurs des tables. Sa base PostgreSQL est distincte de la base de service. Les fichiers Iceberg restent dans MinIO. Spark utilise S3FileIO et le bundle AWS correspondant à Iceberg. Hive active séparément son Hadoop S3A déjà inclus, car il vérifie les chemins externes. Les deux piles Java restent dans des conteneurs distincts.

Le catalogue est initialisé avant Spark. Le schéma Hive n'est créé que s'il est absent. Le pilote JDBC PostgreSQL est inclus dans l'image. Les droits sur les volumes sont préparés à la construction.

La publication entre Iceberg, PostgreSQL et l'export n'est pas une transaction distribuée. Un échec provoque le rejeu du micro-lot. Les clés et les écritures absolues rendent ce rejeu idempotent. Les destinations peuvent cependant être temporairement décalées. La documentation ne promet pas une atomicité globale qui n'a pas été démontrée.

7 Base de service et alertes

PostgreSQL contient les chandeliers courants, les derniers prix et les alertes. La clé des chandeliers est le symbole et le début de minute. Les contraintes imposent les bornes OHLC et un volume non négatif. Les index servent les recherches temporelles et le choix de référence.

Les upserts écrivent des agrégats absolus. Ils n'ajoutent pas le volume d'un lot à une valeur déjà comptée. Une ligne finale ne peut pas être écrasée par une mise à jour provisoire. Un verrou transactionnel sérialise les deux publishers.

Le dernier prix avance selon le couple heure d'événement et identifiant. Une donnée tardive ne le fait pas reculer. À chaque publication live, ce prix est comparé à une observation de cinq minutes auparavant.

La référence est le dernier close dont l'heure du dernier événement est antérieure ou égale à T moins cinq minutes. Elle ne doit pas avoir plus de six minutes. Sans cette référence, aucune alerte n'est créée. Le seuil utilise la valeur absolue de prix divisé par référence moins un. Il doit être strictement supérieur à cinq pour cent.

Le journal conserve le prix, la référence, leurs heures, la variation et l'heure de détection. Une clé par symbole et créneau de cinq minutes empêche les doublons. Un délai minimal de cinq minutes évite les alertes répétitives sur un même mouvement durable.

L'évaluation observe les derniers prix à chaque micro-lot. Un pic bref revenu avant la publication peut être manqué. Une correction tardive peut changer une référence future, mais ne réécrit pas une alerte déjà émise. Ces limites donnent un sens précis au résultat présenté en soutenance.

8 Gold et définitions analytiques

Le raccordement Silver vers Gold utilise une extraction automatisée. Spark lit un snapshot de la table Iceberg par le catalogue. Il écrit un fichier Parquet complet puis remplace atomiquement un pointeur. Dagster capture ce pointeur une fois pour tout le build. Un glob des fichiers data Iceberg serait incorrect, car il pourrait inclure des versions anciennes.

Le modèle gold_minute a le grain action et minute finalisée. Il calcule les moyennes arithmétiques des vingt et cinquante derniers closes minute observés. Sans assez d'observations, le résultat reste NULL. Un trou allonge la durée réelle couverte ; il n'est pas rempli artificiellement.

Le modèle gold_daily a le grain action et jour UTC. Le rendement est la clôture du jour divisée par celle du jour UTC immédiatement précédent, moins un. Sans clôture de la veille, il reste NULL. Le jour courant est marqué partiel.

Le VWAP journalier est la somme des notionals divisée par la somme des volumes. Le calcul conserve donc le poids exact des quantités. Il ne fait pas la moyenne de VWAP minute sans pondération.

La volatilité quotidienne est l'écart type échantillonnal des log-rendements minute consécutifs observés dans le jour. Au moins deux rendements sont nécessaires. Elle reste en unité de rendement par minute et n'est pas annualisée. La méthode est explicite pour éviter de confondre fréquence des observations et grain du résultat.

Les corrélations utilisent Pearson sur les minutes communes des actions, avec au moins trois observations. Le journal Gold des alertes reprend les détections de la base de service. Les modèles sont reconstruits intégralement, choix simple et correct pour ce petit grain et les journées encore modifiées.

9 Qualité et vérifications

La qualité intervient à plusieurs niveaux. Le contrat d'entrée protège Spark contre les événements inutilisables. Les contraintes PostgreSQL empêchent des chandeliers incohérents d'entrer dans la base de service. Les tests dbt vérifient les modèles analytiques. Soda apporte des contrôles opérationnels sur la présence et la fraîcheur.

Les règles imposent low inférieur ou égal à open et close, eux-mêmes inférieurs ou égaux à high. Aucun volume négatif n'est admis. Les champs indispensables ne doivent pas être nuls. La clé symbole et minute doit être unique. La table doit contenir des données et son dernier chandelier doit dater de moins de dix minutes.

Le seuil de fraîcheur tient compte de la finalisation et de la planification Gold. Il n'affirme pas une latence graphique de dix minutes. La mesure transaction vers rendu client serait une autre exigence, classée optionnelle.

Le test métier indépendant fournit un exemple connu : Open 100, High 103, Low 98, Close 103, Volume 10 et Notional 1007. Il inclut un départage d'heure identique et un doublon. Le test Spark reproduit cet exemple dans de vraies requêtes avec état, ajoute des retards puis redémarre les checkpoints.

Le test Gold utilise une base temporaire et soixante minutes simulées. Les valeurs attendues de MA20, MA50 et VWAP sont calculées séparément. L'absence de clôture de la veille ne doit pas produire un rendement inventé. Un jeu incohérent doit faire échouer les contrôles.

Seul le journal VALIDATION.md atteste les contrôles réellement exécutés. Un fichier de configuration, une compilation Python ou une réponse HTTP ne constitue pas une preuve suffisante de toute la chaîne fonctionnelle.

10 Planification et présentation

Dagster pilote le chemin planifié. Une planification s'exécute toutes les cinq minutes. Une autre s'exécute à 00:05 UTC pour les traitements de fin de journée. Elles réutilisent la même fonction, avec un verrou exclusif sur DuckDB. Deux exécutions ne peuvent donc pas écrire simultanément dans le fichier analytique.

Le traitement copie le journal des alertes, construit les modèles dbt et lance Soda. Une publication Gold ne commence que si ces contrôles réussissent. Les trois tables de service changent dans une seule transaction PostgreSQL. En cas d'échec, la dernière version validée reste visible. Dagster conserve l'erreur et effectue deux relances.

Un rapport CSV accompagne les traitements. Le fichier représente les résultats calculés, avec leur grain et leur jour. Il ne contient ni capture fabriquée ni mesure extrapolée.

Grafana est provisionné au démarrage. Sa source PostgreSQL et son tableau de bord ne demandent pas une configuration manuelle. Trois panneaux montrent AAPL, MSFT et NVDA. Le rafraîchissement est de cinq secondes. Les panneaux utilisent les champs Open, High, Low, Close et Volume.

Streamlit propose le choix d'une action, les chandeliers, les indicateurs, le journal des alertes et les corrélations. Les données provisoires sont identifiées. Gold représente les seules fenêtres finalisées, même si la journée courante peut encore être partielle. Les résultats absents ne sont pas remplacés par des chiffres plausibles.

GitHub Actions exécute les tests réalisables sans infrastructure externe. Le workflow versionné ne constitue pas la preuve d'un run distant tant que le dépôt n'est pas poussé. Aucun accès à un compte GitHub ni aucune publication externe n'est supposé.

11 Résultats observés et preuves

Le démarrage complet par Docker Compose réussit sur la machine Windows. Les cinq tests unitaires passent dans l'image Python. Le test Spark réel confirme OHLCV égal à 100, 103, 98 et 103, un volume de 10 et un notional de 1007. Il couvre également les doublons, les retards et la reprise avec checkpoints.

Le contrôle Kafka observe vingt symboles, six partitions et des clés correctes. Il reçoit 5 503 événements en environ vingt-cinq secondes, soit 220,04 par seconde. Les confirmations du producteur restent proches de 220 par seconde, sans erreur. Un lot Bronze complet de 6 681 lignes est relu avec son manifeste.

Une comparaison de 380 lignes finalisées montre des valeurs identiques entre Iceberg et PostgreSQL. La lecture par identifiant de snapshot réussit. Après redémarrage réel de Spark et Bronze, vingt chandeliers finalisés restent inchangés et aucune clé dupliquée n'apparaît.

Le run Dagster du 26 octobre à 15:34 UTC réussit. Il construit six modèles, exécute quinze tests dbt, génère la documentation, passe quatorze contrôles Soda et publie Gold. Un chandelier incohérent injecté dans une base temporaire fait échouer les contrôles comme attendu. Le contrôle de fraîcheur a aussi bloqué un snapshot ancien pendant le rattrapage.

À 15:39 UTC, PostgreSQL contient 2 400 chandeliers, dont 2 340 finalisés, et quinze alertes. Une alerte AAPL à 15:33 UTC mesure une variation de plus de huit pour cent. Grafana affiche trois panneaux de chandeliers. Streamlit affiche le journal, Gold et les corrélations. Les captures proviennent du navigateur réel.

VALIDATION.md conserve les résultats détaillés et leurs limites. Les compteurs évoluent pendant le flux. Aucun run GitHub distant ni déclenchement de fin de journée à 00:05 n'est revendiqué. La consommation ponctuelle observée est d'environ quatre gibioctets pour les services.

12 Limites et soutenance

La plateforme est une maquette locale. Kafka possède un seul broker. PostgreSQL et MinIO utilisent des volumes locaux. La rétention Kafka est limitée. Une panne longue peut dépasser les offsets disponibles. Le code échoue explicitement au lieu d'ignorer cette perte. Une reconstruction historique depuis Bronze nécessite un traitement dédié.

Les checkpoints Spark, la zone Parquet intermédiaire et les tables doivent rester cohérents. Supprimer un seul volume peut invalider les hypothèses de reprise. L'espace disque grandit avec le flux, les fichiers intermédiaires et les snapshots. Une politique de maintenance serait nécessaire pour une exploitation durable.

Le déploiement cible seize gigaoctets physiques. Spark s'exécute localement avec deux threads et une mémoire limitée. Les limites réelles dépendent aussi de la mémoire attribuée à Docker. Le nombre de conteneurs ne suffit pas à évaluer la consommation ; les mesures Docker doivent accompagner une répétition.

L'alerte utilise une observation proche de cinq minutes et une fréquence de micro-lot. Elle ne traite pas tous les ticks intermédiaires. La volatilité décrit les log-rendements minute disponibles. Les moyennes mobiles demandent un historique suffisant. Les champs NULL sont une information correcte, pas un défaut à masquer.

La démonstration dure sept minutes. Elle montre l'ingestion, Bronze, le grain Silver, les chandeliers Grafana, une alerte Streamlit et un run qualité. La préparation doit avoir commencé au moins dix minutes avant ; cinquante minutes sont nécessaires pour observer MA50 sans données historiques ajoutées.

Les bonus de latence complète et de comparaison ClickHouse ne sont pas revendiqués. Le travail privilégie les obligations, la traçabilité et la possibilité de reprendre un traitement sans annoncer des garanties non démontrées.